parcial 2

testing de sw / prueba de software

el desafio del testing es encontrar defectos, eso es lo principal. no asegura ni garantiza calidad, tampoco garantiza q no haya defectos, los hace visibles pero no asegura q no haya mas (q no haya otros escondidos x ahi). la calidad es subjetiva, es la percepcion, y se define como el nivel de cumplimiento de necesidades y expectativas → los requerimientos. se testea cuando ya tenes una parte del producto hecha, no antes

es un proceso destructivo, el objetivo no es probar q el sw anda sino encontrar defectos, x eso se asume q HAY defectos si o si. hay q ir con actitud negativa, arrancar pensando q el sw esta mal. entonces el test es exitoso cuando encuentra defectos (re raro pero asi es, anotar esto q lo toma seguro)

definicion: proceso destructivo de tratar de encontrar defectos (cuya presencia se asume) en el codigo. se va con actitud negativa para demostrar q algo es incorrecto. testing exitoso = el q encuentra defectos.

asegurar vs controlar calidad

una vez q se definen los req de calidad hay q tener en cuenta q la calidad no se puede "inyectar" al final, osea no podes dejar todo para lo ultimo y ponerle calidad ahi. la calidad del producto depende de todas las tareas q se hacen en TODO el proceso, no solo al final. detectar errores temprano ahorra esfuerzo, tiempo y recursos (plata tmb creo). ademas la calidad no es solo del producto sino del proceso y como mejorarlo asi no se repiten los mismos defectos siempre. y el testing NO puede asegurar calidad ni sw de calidad,

defectos vs errores vs falla

el error se descubre con tecnicas especificas dentro de la misma etapa en la q estoy laburando, y siempre es mejor encontrar errores antes q defectos. el defecto es un error q no se detecto y paso a la etapa siguiente, en testing encontramos defectos xq lo incorrecto viene de la etapa anterior (implementacion). la falla es el mal funcionamiento del sistema, pasa como consecuencia de un error o defecto. ojo q no todos los defectos causan fallas, ej los de diseño de interfaz, errores de ortografia, esas cosas no rompen nada

severidad

es la gravedad del defecto. son 5: 1 bloqueante o invalidante → no se puede usar el sistema directamente. 2 critico → no puedo cumplir mi objetivo con esa funcionalidad y no hay plan b, pero igual me queda sistema. 3 mayor → tampoco puedo cumplir el objetivo con esa funcionalidad pero hay forma de solucionarlo (un workaround). 4 menor → no impide la funcionalidad pero hay errores de poca importancia. 5 cosmetico → ortografia, formato de fechas, como estan distribuidos los componentes

prioridad

es la urgencia q tenemos para resolver el defecto. la define el usuario final (si es q lo detecta). niveles: 1 urgente, 2 alta, 3 media, 4 baja. importante: severidad y prioridad son independientes entre si xq dependen del tipo de negocio y de sistema. ej: un defecto cosmetico puede tener prioridad baja en un sistema de inventarios pero alta en una empresa

de marketing donde el aspecto y la presentacion importan un monton (ESTE EJEMPLO ES CLAVE)

niveles de prueba

1 unitarias → las hace el desarrollador. pruebo un componente individual, algo acotado y aislado de lo q estoy desarrollando. como se prueban cosas individuales las ejecuta el mismo dev. aca se encuentran mas errores q defectos, esta en el limite. son muy faciles de automatizar. se hacen en el workflow de implementacion

2 integracion o de interfaces → se integran los componentes q ya pasaron las unitarias para ver como funcionan en conjunto. normalmente hace falta un tester. se integra de forma incremental (de a poco) asi identifico mejor donde estan los errores. tmb en el workflow de implementacion

3 de sistema → mas amplias, no es solo la integracion entre 2 componentes sino el sistema completo, q funcione bien en su totalidad. aca entran tmb los req no funcionales y no solo los funcionales. el ambiente de prueba tiene q ser lo mas parecido posible a produccion. se hacen en el workflow de prueba

4 aceptacion de usuario → workflow de despliegue. las ejecuta el usuario final o el cliente. busca q el cliente se familiarice con el producto y le tenga comodidad y confianza (el objetivo principal NO es encontrar fallas, ojo con eso). el ambiente tmb lo mas parecido a produccion

ambientes para construir el sw

es la configuracion del espacio de trabajo y todas las herramientas o recursos q se necesitan para desarrollar. cada ambiente tiene un proposito y se usa en distintas etapas

desarrollo → donde los devs crean, prueban y depuran el sw. es local o en red. es flexible, pueden probar cosas nuevas sin afectar produccion. aca van las pruebas unitarias

prueba → donde se hacen pruebas exhaustivas (integracion, funcionales, rendimiento y otras). es una replica o parecido a produccion pero sin afectar a los usuarios y sin TODAS las caracteristicas xq es caro tener un ambiente igual al de produccion. usan datos y configuraciones parecidas a produccion. aca van las pruebas de integracion

pre-produccion (tmb le dicen staging o entorno de calidad) → prueba final antes del lanzamiento. se simulan las condiciones de produccion y se hacen pruebas de ultimo minuto: estres, carga, seguridad. el objetivo es validar q el sw este listo para producir sin problemas grandes. aca van las pruebas de sistema

produccion → el sw esta funcionando y lo usan los usuarios finales, es el entorno real. esta muy controlado, es escalable, seguro y confiable, y tiene backup y recuperacion ante desastres. las pruebas de aceptacion se hacen en preproduccion o en un entorno parecido a produccion (o sea el de prueba)

caso de prueba

set de condiciones o variables bajo las cuales el tester determina si el sw funciona bien o no. es ejecutar una serie de pasos sobre una funcionalidad definiendo con q valores voy a hacerlo para ver si me da los resultados esperados. no alcanza con decir q accion ejecutar, tengo q decir tmb con q datos. una buena definicion de casos ayuda a REPRODUCIR defectos. testear sin casos de prueba es perder el tiempo

problema: el testing es caro y no se pueden hacer pruebas exhaustivas (probar todas las combinaciones, imposible). entonces tiene q haber algun mecanismo para definir la menor cantidad de casos posible pero q igual cubran el testing lo mas completo q se pueda. se busca reducir las horas operativas

etapas y artefactos (esto era como un cuadrito)

planificacion de pruebas → plan de pruebas. diseño de casos de prueba → casos de prueba. ejecucion → datos de prueba y version del software. reporte de defectos → reporte de defecto